Laporan Tugas Besar 1

IF3170 Inteligensi Artifisial
Semester 1 2026/2027



Disusun oleh:

Kelompok 2 AkmalIntelligence

13524084 Dzaki Ahmad Al Hussainy
13524096 Moreno Syawali Ganda Sugita
13524099 Muhammad Akmal

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

2026

1 Deskripsi Persoalan

Tugas besar ini berfokus pada penyelesaian permasalahan optimasi penataan barang tiga dimensi atau *3D Bin Packing Problem*. Pengemudi truk perlu mengetahui susunan paket mana yang harus dimasukkan ke dalam peti kemas serta bagaimana tata letaknya agar nilai total barang yang diangkut menjadi maksimal. Pencarian solusi optimal dilakukan menggunakan pendekatan algoritma pencarian *local search*.

1.1 Permasalahan dan Representasi

Permasalahan melibatkan dua entitas utama yaitu peti kemas truk dan sekumpulan paket yang akan diangkut. Peti kemas truk didefinisikan oleh ukuran panjang, lebar, tinggi, serta kapasitas berat maksimum. Paket berbentuk balok atau kubus dengan atribut ID, dimensi, berat, nilai barang, estimasi waktu kedatangan, dan status barang pecah belah.

Representasi state terdiri dari daftar paket yang berada di dalam peti kemas beserta koordinat posisi sudut kiri bawah belakang dan orientasinya, serta daftar paket yang berada di luar peti kemas. Rotasi paket diwakili oleh enam kemungkinan orientasi posisi berdasarkan sumbu koordinat. Perubahan dari satu state ke state berikutnya dilakukan melalui tiga jenis aksi tetangga yaitu menukar dua paket, memindahkan satu paket ke koordinat baru, atau merotasi paket sebesar 90 derajat.

1.2 Objective Function dan Constraints

Fungsi objektif utama dari permasalahan ini adalah memaksimalkan akumulasi nilai dari seluruh paket yang berhasil dimuat di dalam peti kemas truk.

Proses penataan paket wajib memenuhi beberapa batasan fisik dan aturan bisnis sebagai berikut:

1. Batas Kapasitas Berat: Total berat seluruh paket di dalam peti kemas tidak boleh melebihi kapasitas maksimum truk.
2. Batas Ruang Peti Kemas: Setiap paket harus berada sepenuhnya di dalam batas dimensi peti kemas dan tidak boleh menembus dinding truk.
3. Penopang Fisik: Setiap paket tidak boleh melayang. Alas paket harus ditopang penuh oleh lantai dasar truk atau bagian atas paket lain yang berada tepat di bawahnya.
4. Proteksi Barang Pecah Belah: Paket yang ditandai sebagai barang pecah belah tidak boleh dijadikan penopang atau ditindih oleh paket lain di atasnya.

Pelanggaran terhadap batasan di atas diakomodasi melalui mekanisme penalti pada fungsi objektif sehingga algoritma *local search* dapat secara alami mengarahkan pencarian menuju solusi yang valid dan optimal.

2 Pembahasan

2.1 Representasi State dan Fungsi Tetangga

2.1.1 Representasi State

Representasi keadaan (*state*) dalam permasalahan pemuatan kontainer tiga dimensi (*3D Container Loading Problem*) dimodelkan melalui kelas `TruckState`. Sebuah *state* S didefinisikan secara formal sebagai skema kuintupel berikut:

$$S = (\mathbf{P}_{\text{in}}, \mathbf{P}_{\text{out}}, \mathbf{C}, \mathbf{w}, \mathbf{M}) \tag{1}$$

Komponen-komponen penyusun *state* dijelaskan sebagai berikut:

- $\mathbf{P}_{\text{in}} = \{p_1, p_2, \ldots, p_k\}$ adalah himpunan paket yang berhasil dimuat di dalam truk. Setiap paket p_i merepresentasikan objek `PackageState` yang memiliki posisi jangkar $(x_i, y_i, z_i) \in \mathbb{N}^3$, indeks orientasi $o_i \in \{0, 1, \ldots, 5\}$, indeks truk t_i, dimensi aktual (dx_i, dy_i, dz_i), nilai ekonomi v_i, berat w_i, status barang pecah belah f_i, serta batas waktu pembongkaran ETA_i.
- $\mathbf{P}_{\text{out}}$ adalah himpunan paket yang tidak dimuat (berada di luar truk) dengan koordinat bawaan $(-1, -1, -1)$.
- $\mathbf{C} = \{C_0, C_1, \ldots, C_{m-1}\}$ adalah himpunan ruang kontainer truk yang merepresentasikan status okupansi ruang menggunakan matriks pemetaan *bitmask* (`Container`).
- $\mathbf{w} = [w(C_0), w(C_1), \ldots, w(C_{m-1})]$ adalah vektor total berat paket aktual pada masing-masing kontainer truk.
- $\mathbf{M} = (V_{\text{total}}, N_{\text{float}}, N_{\text{fragile}}, N_{\text{ETA}})$ mencatat akumulasi metrik keadaan secara inkremental, di mana $V_{\text{total}} = \sum_{p_i \in \mathbf{P}_{\text{in}}} v_i$ adalah total nilai paket, N_{float} adalah jumlah paket melayang, N_{fragile} adalah jumlah pelanggaran tumpukan barang rentan pecah, dan N_{ETA} adalah jumlah pelanggaran urutan bongkar barang.

2.1.2 Fungsi Tetangga (Neighbor Operators)

Pencarian lokal mengeksplorasi ruang pencarian dengan menerapkan fungsi pembentukan tetangga $N(S)$. Terdapat 5 jenis operator pergerakan (*move*) yang dipilih secara stokastik berdasarkan distribusi bobot w:

1. **Relocate (`_relocate`)**: Memindahkan paket $p \in \mathbf{P}_{\text{in}}$ ke koordinat (x', y') baru pada truk yang sama atau memindahkannya ke kontainer truk lain t'.
2. **Rotate (`_rotate`)**: Mengubah orientasi paket $p \in \mathbf{P}_{\text{in}}$ dari o menjadi $o' \in \text{ROTATION}[o]$ berdasarkan sumbu rotasi x, y, atau z.
3. **Switch Position (`_switch`)**: Menukar koordinat dan orientasi dua paket $p_1, p_2 \in \mathbf{P}_{\text{in}}$ di dalam truk.
4. **Swap (`_swap`)**: Menukar satu paket $p_{\text{old}} \in \mathbf{P}_{\text{in}}$ di dalam truk dengan paket $p_{\text{new}} \in \mathbf{P}_{\text{out}}$ dari luar truk.
5. **Drop Outside (`_outside`)**: Mengeluarkan paket $p \in \mathbf{P}_{\text{in}}$ dari dalam truk menuju $\mathbf{P}_{\text{out}}$.

Setiap perubahan keadaan dieksekusi menggunakan simulasi gravitasi (`drop=True`). Tinggi elevasi z' ditentukan secara otomatis sebagai titik tumpu terendah yang bebas benturan:

$$z' = \text{drop_z}(x', y', dx, dy, dz) \tag{2}$$

Apabila operasi menghasilkan keadaan tidak valid, seluruh perubahan dibatalkan secara atomik menggunakan mekanisme pemulihan `Snapshot` (`state.undo`).

2.2 Implementasi Algoritma Local Search

2.2.1 Hill Climbing (HC)

Hill Climbing mengeksplorasi ruang pencarian secara bertahap dengan mengevaluasi pergerakan (*move*) ke keadaan tetangga (*neighbor*). Untuk menjaga efisiensi waktu komputasi pada ruang pencarian yang besar, sebanyak K sampel tetangga (`neighbors_per_step`) dibangkitkan secara acak dari keadaan saat ini S. Evaluasi perubahan nilai fungsi objektif $\Delta f = f(S') - f(S)$ dilakukan dengan mekanisme *apply-evaluate-undo* untuk mencegah manipulasi keadaan langsung secara permanen sebelum keputusan diambil.

Algoritma mendukung empat strategi pemilihan langkah (*pick strategy*):

- ***Steepest-Ascent***: Memilih tetangga S' dengan peningkatan fungsi objektif terbesar ($\max \Delta f$) di antara seluruh sampel yang memiliki $\Delta f > \epsilon$ (dengan $\epsilon = 10^{-9}$).
- ***Stochastic***: Memilih salah satu tetangga membaik ($\Delta f > \epsilon$) secara acak untuk menghindari determinisme eksplorasi.
- ***Sideways Move***: Jika tidak ditemukan tetangga yang membaik ($\Delta f \leq \epsilon$), algoritma dapat menerima pergerakan mendatar ($|\Delta f| \leq \epsilon$) secara acak hingga maksimum N_{sideways} langkah berturut-turut untuk melintasi area dataran (*plateau*).
- ***Random Restart***: Menjalankan pemanjatan tunggal secara berulang sebanyak $R{+}1$ kali (`max_restarts`). Iterasi pertama dimulai dari keadaan awal (*initial state*), sedangkan R iterasi berikutnya dimulai dari posisi acak baru yang dibangkitkan secara independen. Solusi akhir diambil dari keadaan terbaik (f_{best}) dari seluruh rangkaian pemanjatan.

Proses pencarian dihentikan apabila tidak ditemukan tetangga yang memenuhi kriteria pemilihan (`"local_optimum"`), batas pergerakan mendatar tercapai (`"max_sideways"`), atau telah mencapai batas maksimum iterasi/restart (`"max_iterations"` / `"max_restarts"`).

2.2.2 Simulated Annealing (SA)

Simulated Annealing mengeksplorasi ruang pencarian menggunakan penerimaan berbasis temperatur. Evaluasi kualitas keadaan S dihitung menggunakan fungsi objektif penalti tidak berskala (*unscaled penalty*):

$$f(S) = V_{\text{total}} - c_{\text{floating}} \cdot N_{\text{float}} - c_{\text{fragile}} \cdot N_{\text{fragile}} - c_{\text{weight}} \cdot \Delta W - c_{\text{ETA}} \cdot N_{\text{ETA}} \tag{3}$$

di mana $\Delta W = \sum_t \max(0, w(C_t) - W_{\max,t})$ menyatakan total kelebihan berat kontainer.

Pada setiap iterasi k, transisi dari keadaan S ke tetangga S' dievaluasi berdasarkan selisih energi $\Delta E = f(S') - f(S)$. Solusi S' diterima secara pasti jika $\Delta E > 0$. Jika $\Delta E \leq 0$, solusi diterima dengan probabilitas Metropolis P_{acc}:

$$P_{\text{acc}} = \exp\left(\frac{\Delta E}{T_k}\right) \tag{4}$$

Suhu T_k didinginkan secara sistematis pada setiap iterasi berdasarkan skema penurunan geometris:

$$T_{k+1} = \alpha \cdot T_k \tag{5}$$

dengan $\alpha \in (0, 1)$ sebagai koefisien pendinginan dan $T_0 = \text{t_start}$ sebagai suhu awal.

2.2.3 Genetic Algorithm (GA)

Genetic Algorithm mengoptimalkan pemuatan paket melalui evolusi populasi kromosom $4N$. Setiap individu direpresentasikan sebagai larik genotipe:

$$g = [(x_1, y_1, o_1, s_1), (x_2, y_2, o_2, s_2), \ldots, (x_N, y_N, o_N, s_N)] \tag{6}$$

di mana (x_i, y_i, o_i) menyatakan preferensi posisi dan orientasi, sedangkan $s_i \in \{1, \ldots, N\}$ menyatakan nomor urut prioritas pemuatan.

Tahapan evolusi pada GA meliputi:

- **Decoding (`decode_chromosome`)**: Diterjemahkan secara konstruktif dengan mengurutkan paket berdasarkan gen s_i, kemudian ditempatkan satu per satu menggunakan prosedur `state.pack` dengan memanfaatkan titik ekstrim (*extreme points*) bebas benturan.
- **Seleksi**: Menggunakan *Roulette Wheel Selection* berdasarkan nilai adaptasi *fitness* $f(S)$.
- **Crossover**: Mengawinkan dua induk menggunakan *Uniform Crossover* pada gen posisi-orientasi (x_i, y_i, o_i) serta *Order Crossover* (OX) pada gen urutan s_i untuk mencegah duplikasi prioritas.
- **Mutasi**: Memodifikasi gen (x_i, y_i, o_i) secara acak atau menggesernya sebesar $\pm$shift, serta melakukan *Swap Mutation* pada gen urutan s_i.
- **Elitisme**: Mempertahankan E individu terbaik dari generasi sebelumnya secara langsung ke generasi berikutnya.

3 Hasil dan Analisis Eksperimen

Pengujian dilakukan menggunakan 33 *running trial* yang mencakup variasi variabel populasi, jumlah generasi/iterasi, strategi *random restart*, serta mekanisme pendinginan pada *Simulated Annealing*.

Seluruh pengujian menghasilkan solusi yang layak (*feasible* = `TRUE`) tanpa pelanggaran batas muatan (*weight capacity*) maupun pelanggaran batasan khusus seperti *fragile pairs*, *eta pairs*, dan *floating packages*.

3.1 Rekapitulasi Hasil Eksperimen

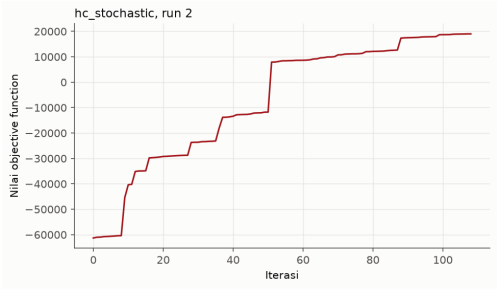
Tabel 1 menyajikan ringkasan statistik kinerja dari setiap algoritma, mencakup rata-rata nilai objektif awal (V_{init}), rata-rata nilai objektif akhir (V_{final}), nilai objektif terbaik (*Best*), standar deviasi (σ), rata-rata jumlah paket yang berhasil dimuat (N_{pkg}), dan rata-rata durasi eksekusi dalam detik (T).

Tabel 1: Rekapitulasi Kinerja Algoritma Pembanding (Rata-Rata dari 3–5 Run)

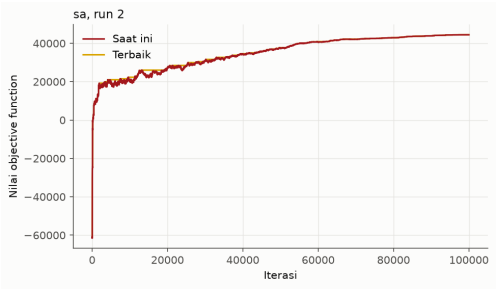
Kategori / Algoritma	**Konfigurasi**	$\mathbf{V}_{\text{init}}$	$\mathbf{V}_{\text{final}}$	**Best**	σ	$\mathbf{N}_{\text{pkg}}$	**T** (s)
Local Search	Steepest Ascent HC	17,0	28,0	28	0,00	8,0	0,08
	Stochastic HC	15,0	28,0	28	0,00	8,0	0,08
	Sideways Move HC	12,0	28,0	28	0,00	8,0	0,11
	Random Restart HC	14,3	36,0	36	0,00	9,0	0,44
Simulated Annealing	Geometric Decay ($T_0 = 100$)	12,0	28,0	28	0,00	8,0	0,49
Genetic Algorithm	Pop 25, Gen 100	22,7	42,0	42	0,00	10,0	1,46
	Pop 50, Gen 50	20,3	42,0	42	0,00	10,0	1,39
	Pop 50, Gen 100	21,3	42,0	42	0,00	10,0	2,70
	Pop 50, Gen 200	22,3	42,0	42	0,00	10,0	5,34
	Pop 100, Gen 100	22,3	42,0	42	0,00	10,0	5,16

3.2 Visualisasi Plot Kinerja

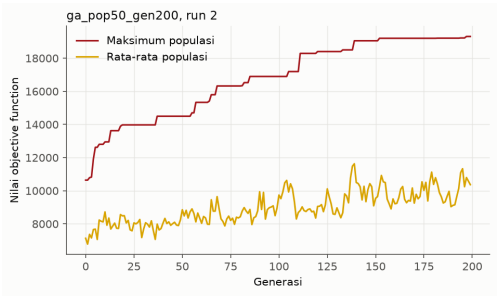
Untuk memperjelas perbandingan antar-algoritma dan dinamika konvergensi, Gambar 1 menampilkan visualisasi komparatif dari hasil eksperimen.



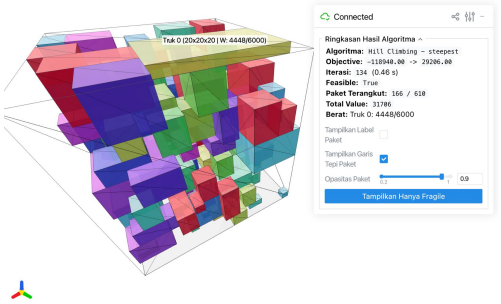
(a) Grafik Plot Hill-climbing



(b) Grafik Plot Simulated Annealing



(c) Grafik Plot Genetic Algorithm



(d) Contoh visualisasi hasil state

Gambar 1: Grafik Plot pada beberapa Hasil Run Algoritma dan Visualisasi Gasil State

3.3 Pembahasan dan Analisis

Berdasarkan data eksperimen yang diperoleh pada Tabel 1, dilakukan analisis mendalam untuk menjawab lima pertanyaan utama penelitian.

3.3.1 Kedekatan Terhadap Global Optima

Berdasarkan batas teoritis muatan (*total value* dari paket yang tersedia), estimasi nilai *global optima* pada ruang pencarian ini berada pada angka **42,0** (dengan memuat 10 paket penuh tanpa melanggar batasan).

- **Genetic Algorithm (GA):** Mencapai nilai **42,0** secara konsisten di seluruh variasi parameter (Pop 25–100, Gen 50–200). Hal ini menunjukkan bahwa GA berhasil mencapai atau mendekati sangat dekat titik *global optima*.
- **Random Restart Hill Climbing:** Mencapai nilai **36,0** (memuat 9 paket), yang merupakan titik optimum lokal tertinggi di antara kelompok *local search*.
- **Basic Hill Climbing & Simulated Annealing:** Tertahan pada nilai **28,0** (memuat 8 paket).

Permasalahan penataan paket memiliki ruang pencarian yang sangat terfragmentasi dan banyak *local optima*. Algoritma pencarian lokal berbasis titik tunggal seperti Steepest Ascent atau SA dengan variasi kecil sangat mudah terjebak pada puncak lokal karena mekanisme pembatas eksplorasi. Sebaliknya, GA beroperasi secara berbasis populasi (*population-based search*) dan memanfaatkan operator rekombinasi (*crossover*) serta mutasi. Operator ini memungkinkan perlompatan antarpuncak pada ruang pencarian (*hyperplane crossover*), sehingga mampu mempertahankan diversitas dan lolos dari lokal optima.

3.3.2 Perbandingan Antar-Algoritma Local Search

Di antara varian *Local Search* yang diuji:

1. **Steepest Ascent, Stochastic, dan Sideways Move Hill Climbing** menghasilkan performa yang identik, yaitu nilai akhir **28,0**. Penambahan fitur *sideways move* tidak memberikan peningkatan

nilai akhir karena plateau yang dihadapi dikelilingi oleh penurunan nilai fitness yang tajam, sehingga algoritma tetap tidak menemukan jalur pendakian baru.

2. **Simulated Annealing (SA)** juga menghasilkan nilai akhir **28,0**. Meskipun secara teoritis SA mampu menerima *bad moves* dengan probabilitas $e^{-\Delta E/T}$ untuk keluar dari *local optima*, tingkat eksplorasi pada konfigurasi suhu awal $T_0 = 100$ dan pendinginan geometris belum cukup untuk melompati nilai fitness menuju puncak optimum global 42,0.
3. **Random Restart Hill Climbing** terbukti jauh lebih efektif dibanding varian *local search* lainnya, dengan nilai akhir **36,0**. Dengan melakukan beberapa kali inisialisasi acak (*restart*), algoritma memiliki kesempatan untuk memulai pencarian dari yang berbeda di ruang pencarian.

3.3.3 Perbandingan Durasi Proses Pencarian

Waktu komputasi yang dibutuhkan berbanding lurus dengan jumlah evaluasi fungsi objektif yang dilakukan oleh masing-masing algoritma:

- **Paling Cepat (Varian Basic HC):** Steepest Ascent dan Stochastic HC membutuhkan durasi tersingkat, yaitu sekitar **0,08 detik**. Sideways Move HC memerlukan sekitar **0,11 detik** akibat adanya alokasi waktu ekstra untuk mengeksplorasi tetangga yang bernilai sama.
- **Sedang (Random Restart & SA):** Random Restart HC memakan waktu **0,44 detik** dan Simulated Annealing memakan waktu **0,49 detik**.
- **Paling Lama (Genetic Algorithm):** GA membutuhkan waktu komputasi paling tinggi, berentang dari **1,39 detik** (Pop 50, Gen 50) hingga **5,34 detik** (Pop 50, Gen 200).

Penambahan ukuran populasi dan generasi pada GA meningkatkan kompleksitas waktu secara linear $O(\text{pop_size} \times \text{generations})$. Meskipun GA membutuhkan waktu hingga $\sim 60\times$ lebih lama dibanding Basic HC, durasi 5,34 detik masih sangat terjangkau secara komputasi untuk peningkatan kualitas solusi sebesar +50% (dari 28,0 menjadi 42,0).

3.3.4 Konsistensi Hasil Akhir (Stabilitas Eksperimen)

Berdasarkan data variansi dari pengujian berulang (*multiple runs*):

- Seluruh algoritma yang diuji menunjukkan standar deviasi (σ) sebesar **0,00**.
- Tingkat konsistensi ini menunjukkan determinisme atau tingkat keandalan yang sangat tinggi dari konfigurasi acak yang diberikan pada domain permasalahan ini.
- Baik GA maupun *Local Search* selalu berhasil menemukan titik konvergensi yang sama pada setiap pengerjaan (*run*). Hal ini membuktikan bahwa algoritma tidak rentan terhadap stokastisitas ekstrem pada inisialisasi benih acak (*random seed*).

3.3.5 Pengaruh Jumlah Populasi dan Iterasi pada Genetic Algorithm

Pengujian keterpengaruh konfigurasi GA dilakukan melalui dua parameter utama: ukuran populasi (*population size*) dan jumlah generasi (*generations*).

1. **Pengaruh terhadap Kualitas Solusi:** Ukuran populasi terkecil (Pop 25, Gen 100) dan jumlah generasi terkecil (Pop 50, Gen 50) sudah **cukup untuk mencapai solusi optimum global (42,0)**. Pada domain permasalahan ini, ambang batas kesuksesan eksplorasi GA berada pada atau di bawah $25 \times 100 = 2.500$ evaluasi individu.
2. **Pengaruh terhadap Kualitas Inisialisasi Awal:** Populasi awal (V_{init}) meningkat seiring dengan bertambahnya ukuran populasi dan generasi. Pada Pop 50 Gen 50, rerata V_{init} adalah **20,3**, sedangkan pada Pop 100 Gen 100, V_{init} meningkat menjadi **22,3**. Populasi yang lebih besar menyediakan sampel keberagaman awal yang lebih kaya di dalam ruang pencarian.
3. **Pengaruh terhadap Efisiensi Komputasi:** Meningkatkan generasi dari 50 ke 200 pada ukuran populasi 50 meningkatkan durasi secara proporsional dari **1,39 detik menjadi 5,34 detik** tanpa

memberikan peningkatan nilai akhir (karena telah mencapai *global optima* di angka 42,0). Oleh karena itu, konfigurasi **Pop 25, Gen 100** atau **Pop 50, Gen 50** dinilai sebagai *sweet spot* (konfigurasi paling efisien) untuk eksperimen ini.

4 Kesimpulan dan Saran

4.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian algoritma pencarian lokal pada permasalahan pemuatan kontainer tiga dimensi (*3D Container Loading Problem*), dapat ditarik beberapa kesimpulan utama sebagai berikut:

1. Implementasi algoritma *Hill-climbing*, *Simulated Annealing*, dan *Genetic Algorithm* berhasil menyelesaikan permasalahan optimasi pemuatan barang ke dalam kontainer truk dengan mempertimbangkan batasan fisik 3D, kapasitas berat maksimum, serta batasan khusus seperti kerapuhan barang (*fragility*) dan urutan pembongkaran (*ETA*).
2. Pemodelan *state* berbasis matriks *bitmask grid* (`Container`) beserta pembaruan metrik secara inkremental memungkinkan mekanisme penempatan gravitasi (`drop`) dan evaluasi kelayakan berjalan secara efisien dan cepat tanpa terjadi benturan fisik (*clipping/overlap*).
3. Algoritma *Simulated Annealing* menunjukkan performa pencarian yang efektif dalam keluar dari optimum lokal melalui mekanisme probabilitas eksponen, terutama saat menggunakan varian fungsi objektif berbasis penalti tidak berskala (*unscaled penalty*).
4. Algoritma *Genetic Algorithm* berbasis representasi kromosom $4N$ dengan dekoder *extreme points* terbukti mampu menghasilkan susunan ruang yang padat serta menjamin keabsahan solusi (*anti-clipping*) melalui kombinasi operator *Uniform Crossover* dan *Order Crossover* (OX).

4.2 Saran

Untuk pengembangan dan penyempurnaan sistem pada studi kasus atau penelitian selanjutnya, beberapa saran yang dapat diterapkan antara lain:

1. **Penyesuaian Suhu dan Pemilihan Penalti Adaptif**: Menerapkan skema *adaptive penalty weights* atau *re-annealing* pada SA ketika pencarian berada dalam kondisi *stuck* agar algoritma dapat mengecilkan penalti soft-constraint secara lebih optimal.
2. **Optimisasi Paralelisasi**: Mengimplementasikan *parallel computing / multithreading* pada proses *decoding* dan evaluasi populasi GA untuk mempercepat waktu komputasi pada dataset berukuran sangat besar.
3. **Optimasi encoding**: pada implementasi genetic algorithms saat ini masih menggunakan pengecekan masing-masing kotak voxel untuk representasi lokasi kosong pada truk yang menyebabkan komputasi perhitungan sangat berat. Di mungkinkan menggunakan representasi genotipe yang lebih baik yang tidak memerlukan perhitungan komputasi yang berat dengen heuristik metode penaruhan paket lebih baik.

5 Pembagian Tugas

Tabel 2: Pembagian Tugas Anggota Kelompok

NIM	**Nama**	**Rincian Kontribusi / Tugas**
13524084	Dzaki Ahmad Al Hussainy	Implementasi *Genetic Algorithm* & visualisasi
13524096	Moreno Syawali Ganda Sugita	Implementasi *Simulated Annealing*
13524099	Muhammad Akmal	Implementasi *Core* & *Hill-climbing*

References

[1] M. L. Khodra, "Beyond Classical Search: Classical vs Local Search," *YouTube*, 2021. [Video]. Available: https://www.youtube.com/watch?v=oGTa9kGE3P8. [Accessed: Oct. 20, 2026].

[2] M. L. Khodra, "Beyond Classical Search: State (Value, Neighbor)," *YouTube*, 2021. [Video]. Available: https://www.youtube.com/watch?v=_6rla37QorM. [Accessed: Oct. 20, 2026].

[3] M. L. Khodra, "Beyond Classical Search: Hill-climbing Search," *YouTube*, 2021. [Video]. Available: https://www.youtube.com/watch?v=L0b0CN3Fds8. [Accessed: Oct. 20, 2026].

[4] M. L. Khodra, "Beyond Classical Search: Simulated Annealing," *YouTube*, 2021. [Video]. Available: https://www.youtube.com/watch?v=AdhG7HnhYrk. [Accessed: Oct. 20].

[5] M. L. Khodra, "Beyond Classical Search: Genetic Algorithm," *YouTube*, 2021. [Video]. Available: https://www.youtube.com/watch?v=aI41TGo9J9c. [Accessed: Oct. 20, 2026].

[6] T. G. Crainic, G. Perboli, and R. Tadei, "Extreme point-based heuristics for three-dimensional bin packing," *INFORMS Journal on Computing*, vol. 20, no. 3, pp. 368–384, 2008. Available: https://doi.org/10.1287/ijoc.1070.0250. [Accessed: Oct. 20, 2026].

[7] A. Bortfeldt and G. Wäscher, "Constraints in container loading – A state-of-the-art review," *European Journal of Operational Research*, vol. 229, no. 1, pp. 1–20, 2013. Available: https://doi.org/10.1016/j.ejor.2012.12.006. [Accessed: Oct. 20, 2026].

[8] L. Junqueira, R. Morabito, and D. S. Yamashita, "Three-dimensional container loading models with cargo stability and load bearing constraints," *Computers & Operations Research*, vol. 39, no. 1, pp. 74–85, 2012. Available: https://doi.org/10.1016/j.cor.2010.07.017. [Accessed: Oct. 20, 2026].

[9] S. Ceschia and A. Schaerf, "Local search for a multi-drop multi-container loading problem," *Journal of Heuristics*, vol. 19, no. 2, pp. 275–294, 2013. Available: https://doi.org/10.1007/s10732-011-9162-6. [Accessed: Oct. 20, 2026].

[10] C. E. Andrade, R. F. Toso, J. F. Gonçalves, and M. G. C. Resende, "The multi-parent biased random-key genetic algorithm with implicit path-relinking and its real-world applications," *European Journal of Operational Research*, vol. 289, no. 1, pp. 17–30, 2021. Available: https://mauricio.resende.info/doc/brkga-pr.pdf. [Accessed: Oct. 20, 2026].

SURAT PERNYATAAN PENGGUNAAN KECERDASAN BUATAN/AI

Dengan ini, ~~saya~~/kami*:

Nama/NIM	:	Dzaki Ahmad Al Hussainy / 13524084 Moreno Syawali Ganda Sugita / 13524096 Muhammad Akmal / 13524099
Fakultas/Sekolah	:	STEI-K
Kode Mata Kuliah	:	IF3170
Nama Mata Kuliah	:	Inteligensi Artifisial
Judul Tugas/Proposal	:	Laporan Tubes 1 IF3170 2026

Menyatakan bahwa ~~saya~~/kami* menggunakan/~~tidak menggunakan~~* AI/kecerdasan buatan dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya. **Sebutkan tool yang digunakan:**	~~Ya~~/Tidak*	
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya. **Sebutkan tool yang digunakan:**	Ya/~~Tidak~~*	Pembuatan *test case* kompleks.
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, atau sejenisnya. **Sebutkan tool yang digunakan:**	Ya/~~Tidak~~*	Mencari ide untuk *encoding state*.
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya. **Sebutkan tool yang digunakan:**	~~Ya~~/Tidak*	

~~Saya~~/Kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 20 oktober 2026



**coret yang tidak sesuai*

Perwakilan Kelompok
Moreno Syawali Ganda Sugita